WORD VECTORS

NATURAL LANGUAGE PROCESSING LECTURE NOTES

Yue Ning
Stevens Institute of Technology
yue.ning@stevens.edu

1 Introduction

Traditional NLP methods for representing word meaning have relied on resources such as WordNet, which maintain synonyms and hypernyms. However, these resources suffer from several limitations.

- They may lack nuances in meaning. For example, "proficient" is listed as a synonym for "good", but this is not always accurate.
- They may not include new word meanings, such as "wicked", "nifty", "ninja", or "bombest".
- Organizing words in these resources is subjective and requires significant human effort to maintain and update.
- Computing word similarity using these resources is not straightforward.

1.1 Discrete symbols

In traditional NLP, words are represented as discrete symbols, such as one-hot vectors:

- motel = $[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]$
- mhotel = $[0, 1, 0, 0, 0, 0, 0, 0, 0, 0]$

The dimension of these vectors is equal to the number of words in a pre-defined vocabulary (e.g., 500,000).

A key problem with discrete symbols is that their similarity computations result in orthogonal vectors, meaning their inner product is zero. Using WordNet's synonym lists to determine similarity is not effective due to incompleteness.

Note: In information retrieval, we can use discrete symbols (vectors) to represent a document (i.e., bag of words). One popular accepted metric is Term Frequency - Inverse Document Frequency (TFIDF) [4].

2 Word2vec - Skip-Gram (SG)

To address the limitations of discrete word representations, Word2Vec provides a solution by learning continuous vector representations that encode word similarities. The skip-gram model [2], a variant of Word2Vec, learns these dense vectors by predicting surrounding words given a target word.

Given a large corpus of text, we go through each position t in the text, which has a center word c and context words o. We use the similarity of word vectors (e.g., inner product) for c and o to calculate the probability of o given c or vice versa. The process keeps adjusting the word vectors to maximize this probability.

For each position $t = 1, ..., T$, the model predicts context words within a window of fixed size m, given a center word w_t. The objective is to maximize the likelihood of all words in all the windows where θ denotes all the word vectors:

$$\text{Likelihood} = L(\theta) = \prod_{t=1}^{T} \prod_{\substack{j \in [-m,m] \\ j \neq 0}} P(w_{t+j} | w_t; \theta), \tag{1}$$

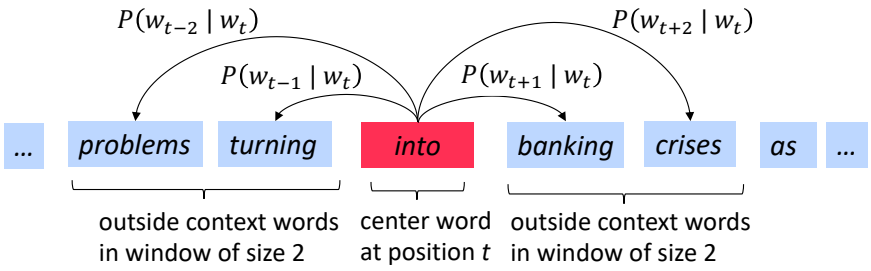

Figure 1: One example window and the process for computing $P(w_{t+j}|w_t)$. Figure source: [1]

where $P(w_{t+j}|w_t;\theta) \in \mathbb{R}^{|V|}$ is a probability distribution over all unique words in your vocabulary.

We then transform this maximization problem into a minimization problem by taking the (average) negative logarithm of the likelihood:

$$J(\theta) = -\frac{1}{T}\log L(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{j\in[-m,m]\\ j\neq 0}}\log P(w_{t+j}|w_t;\theta). \tag{2}$$

Then the next question is how to calculate $P(w_{t+j}|w_t;\theta)$? We will use two vectors per word w:

- $\mathbf{v}_w$ when w is a center word
- $\mathbf{u}_w$ when w is a context word

Then for a center word c and a context word o, the probability of the context word appearing in the window of the center word is defined as follows (**inner product of two vectors signifies the similarity of these two vectors**):

$$P(o|c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w\in V}\exp(\mathbf{u}_w^\top \mathbf{v}_c)}. \tag{3}$$

This is the softmax function which maps arbitrary values x_i to a probability distribution p_i:

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^{n}\exp(x_j)}. \tag{4}$$

This function has "max" because it amplifies the probability of largest x_i, and it is "soft" because it still assigns some probability to smaller x_j. It is frequently used in deep learning.

After we define the loss function, we can apply gradient descent to estimate/learn the word vectors by treating them as model parameters.

Gradient Descent for word2vec

- Initialize θ (all word vectors)
- For k in $1, ..., K$ (epochs):
  - Calculate the total training loss J and its gradient with respect to θ : $\frac{\partial J(\theta)}{\partial \theta}$
  - Update θ: $\theta^{k+1} = \theta^k - \eta\frac{\partial J(\theta)}{\partial \theta}$

In reality, the calculation of $\frac{\partial J(\theta)}{\partial \theta}$ can be expensive. When we have a large corpus, gradient descent can be slow due to that the loss is calculated based on the sum of all the windows. We can apply stochastic gradient descent to accelerate the learning process by treating each window as a sample. So gradient/loss will be computed based on one sample each time.

Stochastic Gradient Descent for word2vec

- Initialize θ (all word vectors)
- For k in $1, ..., K$ (epochs):
  - Sample a window x_i
  - Calculate the loss J of the current window (x_i) and its gradient with respect to θ on this single sample: $\frac{\partial J(x_i,\theta)}{\partial \theta}$
  - Update θ: $\theta^{k+1} = \theta^k - \eta \frac{\partial J(x_i,\theta)}{\partial \theta}$

2.1 Derivation of gradient for center word vector

In this section, we show how we derive the gradient of $J(\theta)$ w.r.t $\mathbf{v}_c$ and you can also derive the gradient for $\mathbf{u}_o$. First, we know the loss function over the training set is defined as below:

$$J(\theta) = -\frac{1}{T}\log L(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{j\in[-m,m]\\ j\neq 0}}\log P(w_{t+j}|w_t;\theta). \tag{5}$$

The gradient of J w.r.t $\mathbf{v}_c$, $\frac{\partial J}{\partial \mathbf{v}_c}$, is equal to $-\frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{j\in[-m,m]\\ j\neq 0}}\frac{\partial \log P(w_{t+j}|w_t;\theta)}{\partial \mathbf{v}_c}$. So we just need to calculate the below part:

$$\frac{\partial \log P(w_{t+j}|w_t;\theta)}{\partial \mathbf{v}_c} \tag{6}$$

where $P(w_{t+j}|w_t;\theta) = P(\mathbf{u}_o|\mathbf{v}_c)$ which means:

$$\log P(\mathbf{u}_o|\mathbf{v}_c) = \log\frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w=1}^{V}\exp(\mathbf{u}_w|\mathbf{v}_c)} = \mathbf{u}_o^\top \mathbf{v}_c - \log\sum_{w=1}^{V}\exp(\mathbf{u}_w^\top \mathbf{v}_c) \tag{7}$$

So the gradient $\frac{\partial \log P(w_{t+j}|w_t;\theta)}{\partial \mathbf{v}_c}$ is calculated as follows:

$$\frac{\partial \log P(w_{t+j}|w_t;\theta)}{\partial \mathbf{v}_c} = \frac{\partial \log \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w=1}^{V}\exp(\mathbf{u}_w^\top \mathbf{v}_c)}}{\partial \mathbf{v}_c} \tag{8}$$

$$= \frac{\partial\left[\mathbf{u}_o^\top \mathbf{v}_c - \log\sum_{w=1}^{V}\exp(\mathbf{u}_w^\top \mathbf{v}_c)\right]}{\partial \mathbf{v}_c} \quad \text{because } \log\frac{A}{B} = \log A - \log B \tag{9}$$

$$= \mathbf{u}_o - \frac{1}{\sum_{w=1}^{V}\exp(\mathbf{u}_w^\top \mathbf{v}_c)}\cdot\frac{\partial\sum_{x=1}^{V}\exp(\mathbf{u}_x^\top \mathbf{v}_c)}{\partial \mathbf{v}_c} \quad \text{applying chain rule} \tag{10}$$

$$= \mathbf{u}_o - \frac{1}{\sum_{w=1}^{V}\exp(\mathbf{u}_w^\top \mathbf{v}_c)}\cdot\sum_{x=1}^{V}\frac{\partial\exp(\mathbf{u}_x^\top \mathbf{v}_c)}{\partial \mathbf{v}_c} \quad \text{moving derivation inside sum} \tag{11}$$

$$= \mathbf{u}_o - \frac{1}{\sum_{w=1}^{V}\exp(\mathbf{u}_w^\top \mathbf{v}_c)}\cdot\sum_{x=1}^{V}\exp(\mathbf{u}_x^\top \mathbf{v}_c)\cdot\mathbf{u}_x \quad \text{applying chain rule} \tag{12}$$

$$= \mathbf{u}_o - \sum_{x=1}^{V}\frac{\exp(\mathbf{u}_x^\top \mathbf{v}_c)\cdot\mathbf{u}_x}{\sum_{w=1}^{V}\exp(\mathbf{u}_w^\top \mathbf{v}_c)} \quad \text{moving the factor inside} \tag{13}$$

$$= \underbrace{\mathbf{u}_o}_{\text{observed}} - \underbrace{\sum_{x=1}^{V}p(\mathbf{u}_x|\mathbf{v}_c)\cdot\mathbf{u}_x}_{\text{expectation}} \quad \text{definition of expectation} \tag{14}$$

This method is called the Skip-gram (SG) model. If we predict center words from context words, it is Continuous Bag of Words (CBOW).

2.2 Negative Sampling

The normalization factor is too computationally expensive (sum of $\mathbf{u}_w^\top \mathbf{v}_c$ for all the words in the vocabulary):

$$P(o|c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in V} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \tag{15}$$

Hence, in standard word2vec, we implement the skip-gram model with **negative sampling** [3]. The main idea is to train binary logistic regressions for a true pair (center word and word in its context window) versus several noise pairs (the center word paird with a random word). The overall objective function to maximize is defined as below:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J_t(\theta), \tag{16}$$

$$J_t(\theta) = \log \sigma(\underbrace{\mathbf{u}_o^\top \mathbf{v}_c}_{\text{true pair}}) + \sum_{i=1}^{k} \mathbb{E}_{j \sim P(w)} [\log \sigma(- \underbrace{\mathbf{u}_j^\top \mathbf{v}_c}_{\text{noise pair}})], \tag{17}$$

where $P(w) = U(w)^{3/4}/Z$ which is the unigram distribution of words $U(w)$ to the power of $3/4$. The power allows less frequent words be sampled more often.

Using this objective function, we can maximize the probability of two words co-occurring in the first log while minimizing the probability of two words co-occurring in the second log (negative samples that two words do not co-occur in the same context).

References

[1] C. Manning and J. Hewitt. CS224N: Natural language processing with deep learning.

[2] T. Mikolov, K. Chen, G. S. Corrado, and J. Dean. Efficient estimation of word representations in vector space, 2013.

[3] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean. Distributed representations of words and phrases and their compositionality. In C. J. C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 26*, pages 3111–3119. Curran Associates, Inc., 2013.

[4] K. Sparck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28:11–21, 1972.